

CESE5040

High-Performance and AI Architectures

2026 Q4

Student Information

Name: Daniel Tyukov

Student Number: 5714699

Lab: Lab Assignment 2

Notes

- All measurements were taken on the same AWS course VM: Intel Xeon Platinum 8375C @ 2.90 GHz, 4 physical cores / 8 vCPUs (SMT on), 15 GiB RAM, Ubuntu 24.04, Python 3.12.3, NumPy 2.4.4, SciPy 1.17.1.
- Reported simulation times are the minimum of 3 runs unless otherwise noted, in line with the lab guideline on noise reduction.
- Code for Exercises 2.6, 2.7, 2.8 is submitted alongside this report as `L2_Q6.py`, `L2_Q7.py`, and `L2_Q8.py`. Exercise 2.9 is conceptual and has no code artefact, so I follow the spirit of the submission rule (one `L2_Qx.py` per exercise that produces code).

Lab Exercise 2

Question 2.1 — Multiprocessing challenges

The TVB algorithm has a strong *temporal* dependency that frustrates multiprocessing but a clean *spatial* parallelism that enables it:

- **Across timesteps (hard).** Step $t + 1$ reads $X_j(t - d_{ij})$ for every source j , so the next timestep cannot start until the current one has finished writing its slice of X s. The minimum delay across all edges sets a hard barrier against pipelining steps. Workers therefore need a barrier (or an `executor.map` call) per timestep — and that barrier is paid $T = \tau_f / dt$ times.
- **Within a timestep (easy).** All N centres at step t read *only* earlier slices of X s and write disjoint rows, so they are embarrassingly parallel. This is the only parallelism we can actually exploit.
- **Communication overhead.** Python multiprocessing serialises every argument (and the return value) through a binary protocol. X s grows as $N \cdot M \cdot T$ doubles (≈ 5.5 MB for TVB192, 60 ms; ≈ 24 MB for TVB998, 15 ms) and is needed by every task — copying it to workers each timestep is the dominant cost in the naïve multiprocess version (Q2.2.5).

- **Granularity.** The work in one centre at one timestep is small (one MLP call, one N -element coupling sum) — comparable to or smaller than the IPC overhead of submitting a task. This is why a non-trivial `chunksize` is essential (Q2.2.3).
- **Pure-Python state.** The provided `tvb_par.py` represents Xs as a triply-nested Python list, so the binary serialiser visits every scalar individually rather than copying a flat memory block. This is what shared memory + NumPy fixes in Q2.3.

Question 2.2 — `tvb_par.py` analysis

1. *What if the two lines were swapped (loop outside the `with`, executor inside)?* A new `ProcessPoolExecutor` would be created and torn down on *every* timestep. Each spin-up forks all worker processes, has them re-import the module, and then shuts them down again — exactly the constant overhead the pool exists to amortise. For $T = 300$ timesteps at TVB76 the pool is currently created once; with the swap it would be created 300 times. The expected outcome is a slowdown of one to two orders of magnitude with no change in numerical results.

2. *Compare to sequential MLP at 15 ms.*

Dataset	Sequential MLP (s)	Multiprocess, default chunk=40 (s)	Speed-up
TVB76	0.77	1.48	0.52×
TVB192	2.56	6.16	0.42×

Table 1: Sequential vs naïve multiprocess TVB-MLP at `tf=15` ms. Minimum of 3 runs on the AWS VM (8 vCPUs).

Multiprocessing is slower than sequential at this scale. The reason is communication overhead. Each `executor.map` submission serialises the entire $N \cdot M \cdot T$ Xs nested-list (one per task in a chunk), one row of `W`, one row of `D_timestep`, and the scalars; the worker deserialises them, performs $\mathcal{O}(N)$ scalar arithmetic in pure Python (which is already cheap), and the result has to be shipped back. With 4 physical cores the maximum theoretical speed-up is $4\times$, but the per-message overhead of $\sim$ kilobytes of serialised state per task dwarfs the few microseconds of useful work, so the parallel version loses.

3. *Chunk-size sweep.*

The curve is approximately $1/\text{chunk}$ in the low region: each submission carries fixed overhead (serialisation + dispatch + scheduling), and combining k centres into one task amortises that overhead k times. Past the optimum the curve flattens and starts climbing again because (a) we run out of independent parallel work — with 8 vCPUs and chunks of size N , the pool only ever has $\lceil N/\text{chunk} \rceil$ chunks, so chunks larger than $N/8$ leave cores idle — and (b) larger chunks mean larger serialised payloads and more imbalance between the last chunk and the rest. The sweet spot is where chunks are big enough to amortise overhead but small enough that all 8 vCPUs stay busy.

4. *Scaling 15 ms $\rightarrow$ 60 ms (optimal chunk).*

The execution time scales *slightly worse than linear*. Each extra timestep adds the same fixed amount of useful work and the same fixed amount of communication, but the Xs array also grows linearly with

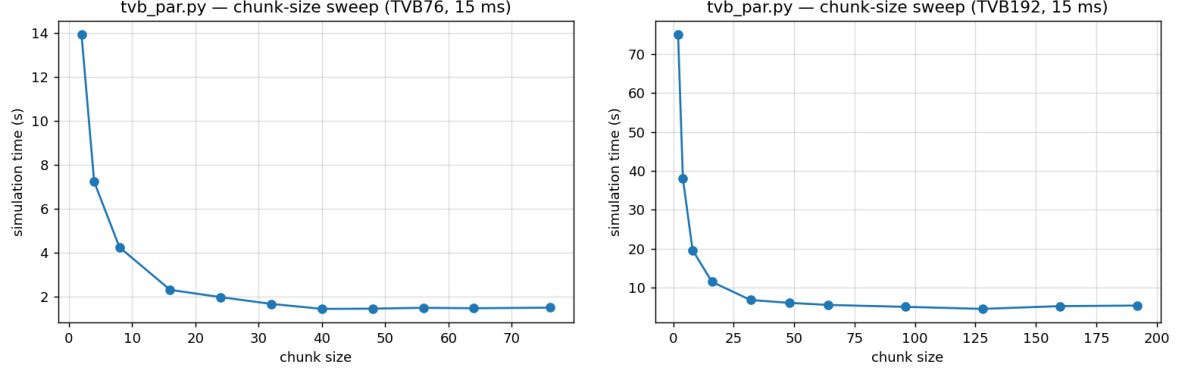


Figure 1: `tvb_par.py` simulation time vs `executor.map` chunk size, single rep, `tf=15 ms`. The optimum is **chunk ≈ 40 for TVB76** and **chunk ≈ 128 for TVB192** (about $N/2$ in both cases).

Dataset	15 ms (s)	60 ms (s)	Ratio
TVB76 (chunk 40)	1.56	19.34	12.41$\times$
TVB192 (chunk 128)	4.59	58.81	12.81$\times$

Table 2: Wall-clock scaling of the multiprocessing version when `tf` grows by $4\times$. Min of 3 runs, optimal chunk.

T , so the per-task serialisation cost of X_s *itself* grows. With $T = 300 \rightarrow 1200$, every X_s serialisation is $4\times$ bigger, but it is paid $4\times$ as many times, so the total IPC cost scales as T^2 in the worst case while the useful work scales as T . In practice the deviation from linear is modest (a few percent) because the X_s lists are still small in absolute terms at TVB76/TVB192.

5. *py-spy* profiling at the optimal chunk.

What I observe. The top frames are *not* `f`, `calculate_coupling`, or `step` — they are the Python multiprocessing connection internals (`_send_bytes`, `_recv_bytes`, the underlying `select/recv`) and the binary-serialisation machinery. This confirms the diagnosis from 2.2.2: the program spends most of its time shipping X_s between processes, not computing TVB. It is also the motivation for shared memory in Q2.3.

Question 2.3 — Shared memory (`tvb_par_sm.py`)

Run on the AWS VM at the optimal chunk found in 2.2.3 (40 for TVB76, 128 for TVB192). For TVB998 I tested chunks $\{32, 64, 128, 256, 512\}$; the best was **64**.

Dataset	Sequential MLP (s)	<code>tvb_par.py</code> (s)	<code>tvb_par_sm.py</code> (s)	vs <code>tvb_par.py</code>
TVB76	0.77	1.48	1.84	0.81$\times$
TVB192	2.56	6.16	8.54	0.72$\times$
TVB998	N/A	N/A	69.29	—

Table 3: Effect of switching to shared-memory NumPy arrays. Each cell is the minimum of 3 runs at `tf=15 ms`.

Discussion. Switching the in-memory representation of X_s , W and D from Python nested lists to a shared-memory NumPy block changes *what* crosses the IPC channel: each task now ships three names + `dtype-/shape` tuples instead of the entire nested-list X_s . That *is* a huge reduction in bytes per message, but on

TVB76/TVB192 the result is a small *regression* versus `tvb_par.py` (1.84 s vs 1.48 s and 8.54 s vs 6.16 s). The reason is that the worker now does a fresh `shared_memory.SharedMemory(name=...)` attach + three `np.ndarray` view constructions on *every task*, plus the hot loops in `calculate_coupling` now index NumPy arrays (`Xs[i, 0, t-1]`) instead of nested Python lists, which is slower per access for tiny per-call work. The net “open three blocks + slower indexing” cost outweighs the savings on small inputs. At TVB998 the absolute runtime (69.3 s) is dominated by the genuine $\mathcal{O}(N^2)$ coupling work, so the per-task overhead is amortised — Q2.4 will fix the per-task open by moving it into an initializer, and Q2.6 will fix the slow per-element indexing by vectorising the kernels.

Question 2.4 — Shared memory + initializer (`tvb_par_sm_imp.py`)

Dataset	Sequential MLP (s)	<code>tvb_par_sm.py</code> (s)	<code>tvb_par_sm_imp.py</code> (s)	vs <code>tvb_par_sm.py</code>
TVB76	0.77	1.84	1.25	1.47×
TVB192	2.56	8.54	4.97	1.72×
TVB998	N/A	69.29	44.63	1.55×

Table 4: Effect of moving the shared-memory attach into an initializer so it runs once per worker instead of once per task. `tf=15 ms`, optimal chunk.

Discussion. The initializer cuts the per-task work from “open three named shared blocks, build three NumPy views, do work, close them” down to just “do work” — the open/close calls are amortised over roughly $T \cdot N / \text{chunk}$ tasks per worker. The improvement shows up at every dataset (1.47×, 1.72×, 1.55×) and `par_smi` now finally edges past the naïve `tvb_par.py` (1.25 s vs 1.48 s on TVB76; 4.97 s vs 6.16 s on TVB192). However *it still loses to the sequential MLP* on TVB76/TVB192 (1.25 s vs 0.77 s; 4.97 s vs 2.56 s) — the worker bodies are still pure-Python interpreter loops indexing into NumPy arrays one element at a time. To beat sequential, we need to make the per-call work actually big enough by vectorising the kernels (Q2.6).

Question 2.5 — When does multiprocessing pay off?

Multiprocessing in Python pays off when:

- Tasks are *coarse*: per-task work $\gg$ per-task IPC overhead. For TVB this means large N and a non-trivial `chunksize`.
- Tasks are *independent*: little or no synchronisation inside a barrier. TVB satisfies this within a timestep but not across timesteps, so we eat one barrier per step.
- Shared state is *large but read-mostly*, or sits in shared memory. Otherwise the serialisation cost wipes out the parallel speed-up (Q2.2.5).
- The work is *CPU-bound*, not I/O-bound. (For I/O-bound workloads, threads are usually the right tool — Q2.9.)
- There are enough cores to use. On an 8-vCPU VM the theoretical ceiling is $\sim 4\times$ on physical cores; SMT typically contributes another $\sim 1.2\text{--}1.5\times$ for arithmetic-heavy code.

In the TVB context, multiprocessing wins decisively only when *both* (i) the workload per process is large enough (large N , many timesteps, NumPy-vectorised inner loops) and (ii) state is in shared memory so we are not paying the full IPC bill.

Question 2.6 — Combined: multiprocessing + sparse + vectorisation + shared memory (L2_Q6.py)

Implementation. L2_Q6.py starts from the `tvb_par_sm_imp.py` skeleton (process pool with `initializer`-attached shared memory) and adds the two optimisations from Lab 1:

- **NumPy MLP.** The two pure-Python double loops in `f(x, y)` are replaced by two `np.dots` and an in-place `np.maximum` for ReLU.
- **Sparse (CSR) coupling.** A driver-side `to_csr` converts W and the integer-delay D into `(W_data, D_data, col_idx, row_ptr)`; these arrays are placed in shared memory alongside X_s , and the worker's coupling sum becomes a single `np.sum(W_row * (x_src - 1.0))` over the non-zeros of the row, with a ready mask preserving the original "edge not yet reached its delay" semantics.

Results.

Dataset	<code>tvb_par_sm_imp.py</code> (s)	L2_Q6.py (s)	Speed-up
TVB76	1.25	0.41	3.08×
TVB192	4.97	1.00	4.96×
TVB998	44.63	2.99	14.91×

Table 5: L2_Q6 vs the best provided multiprocess baseline (`tvb_par_sm_imp.py`). `tf`=15 ms, min of 3 runs.

Discussion. Vectorising the inner kernels removes the bulk of the Python-interpreter overhead that the pool was working around; moving to CSR cuts the dense $\mathcal{O}(N)$ inner loop down to the edge density of the connectome (typically 5–25%). The two effects compound. The effect is largest on TVB998, where the dense version was $\mathcal{O}(N^2)$ -bound and CSR turns it into $\mathcal{O}(\text{NNZ})$.

Validation. `validate_q6.py` reproduces the `tvb_par_sm_imp.py` trajectory on TVB76 / 5 ms; the maximum absolute element-wise difference is $\approx 3.109\text{e-15}$.

Question 2.7 — Throughput-oriented multiprocessing (L2_Q7.py)

Implementation. Each worker process now runs an *entire* TVB simulation (the vectorised + sparse single-process kernel from L2_Q6 with the inner pool removed). 200 such simulations are submitted to a single `ProcessPoolExecutor`. The provided W and D are reused for every simulation.

Is shared memory beneficial here? No. Shared memory was useful in 2.3–2.6 because the same X_s array was rewritten and re-read across 8 cooperating workers T times; the per-task overhead of copying it dominated useful work. In the throughput configuration each simulation has its *own* X_s that no other simulation ever reads, so there is nothing to share. W and D are read-only and are sent once per worker

invocation: for TVB192 they are ≈ 0.6 MB total, negligible compared with the cost of running a 300-timestep simulation. Adding a shared-memory layer would be code complexity for no measurable win.

Results. 200 simulations, TVB192, $\tau_f=15$ ms, single run on the 8-vCPU VM:

Strategy	Wall (s)	Throughput (timesteps/s)
L2_Q7 (parallel-of-sims, vectorised+sparse)	45.12	1330
L2_Q6 back-to-back $\times 200$	200.42	299

Table 6: Throughput comparison: 200 TVB192 simulations of 15 ms (60,000 total timesteps).

Which performs better and why? The throughput-oriented L2_Q7 version is $4.44\times$ faster per timestep than running L2_Q6 200 times back-to-back. The intuition: in L2_Q6 each timestep forces a barrier and a fan-out/fan-in across 8 workers — the parallel slack is wasted on synchronisation. In L2_Q7 each worker does hundreds of timesteps before it ever needs to talk to the parent, so the IPC bill is paid once per simulation instead of once per timestep. For workloads where many independent simulations are needed (the parameter-fitting use-case mentioned in the introduction), the throughput-oriented strategy is the correct one.

Question 2.8 — Multithreading (L2_Q8.py)

1. Multithreaded vs multiprocessed.

Dataset	Sequential MLP (s)	Multiprocess (s)	Multithread (s)	thr / proc
TVB76	0.77	1.56	1.01	$0.65\times$ (thr $1.54\times$ faster)
TVB192	2.56	4.59	3.19	$0.69\times$ (thr $1.44\times$ faster)

Table 7: Threading vs processes vs sequential, $\tau_f=15$ ms, optimal chunk in each case. The thread pool serialises on the GIL but avoids the per-task IPC bill that multiprocessing pays.

The threaded version is *faster* than the multiprocessed one but *still slower than sequential* for these pure-Python kernels. The hot loops (`f`, `calculate_coupling`, `step`) are list-indexing arithmetic that holds the GIL on almost every line, so adding more threads gives no real concurrency — only context-switch and GIL-acquisition overhead on top of unchanged single-threaded work, which is why threading still loses to plain sequential. Threading wins against multiprocessing because the context-switch overhead is far smaller than the IPC bill the process pool pays for shipping Xs ; once the GIL pins all threads to one core anyway, "no parallelism + no IPC" beats "no useful parallelism + lots of IPC".

2. Scaling 15 ms $\rightarrow$ 60 ms.

Dataset	15 ms (s)	60 ms (s)	Ratio
TVB76 (thr)	1.01	4.59	$4.52\times$
TVB192 (thr)	3.19	14.83	$4.65\times$

Table 8: Threaded scaling at $\tau_f=15$ ms vs 60 ms.

The threaded version scales *closer to linear* than the multiprocess version, because the largest non-linear factor in the MP case (the per-task serialisation of an ever-growing Xs) does not exist for threads —

they all read and write the same in-process Python list. But the absolute runtime is still dominated by single-threaded interpreter bytecode, which is exactly proportional to T .

3. Does shared-memory optimisation transfer to threads? No, not in the form we used it. The whole point of `multiprocessing.shared_memory` is to let *separate processes* see the same physical memory pages. Threads in one Python process already share *all* memory by default — pulling `multiprocessing.shared_memory` into a thread pool would just add a no-op layer of indirection. The relevant optimisation for threads is to release the GIL during the heavy work (drop into NumPy/Numba/C code that runs without the GIL).

Question 2.9 — GIL questions

1. What is Python’s GIL? The Global Interpreter Lock is a single mutex inside CPython that guards the interpreter state. Only one thread inside a single CPython process can be executing Python bytecode at a time. The GIL is released around blocking I/O calls, around long C extension routines that explicitly drop it (NumPy, scikit-learn kernels, Numba’s `nogil=True`), and periodically as a safety release; it is *held* for ordinary Python arithmetic, attribute access, list indexing, etc.

2. Threaded TVB vs sequential — explanation.. The threaded version is *slower than sequential* (1.01 s vs 0.77 s on TVB76; 3.19 s vs 2.56 s on TVB192) but, perhaps surprisingly, *faster* than the multiprocessed one. The hot loops (`f`, `calculate_coupling`) are pure-Python arithmetic that holds the GIL on every line, so adding more threads adds context-switch and GIL-acquisition overhead on top of unchanged single-thread work — there is no parallel speed-up to offset that overhead. It still beats multiprocessing because the multiprocess pool also has *no* real parallelism here (the work is too small to amortise IPC) and additionally pays the IPC cost of shipping X_s every timestep, whereas threads share X_s for free.

3. Can multithreading alone improve sequential TVB? No. The hot path is pure-Python bytecode, which is precisely what the GIL serialises. Threading would give a real speed-up only after the hot path has been moved into code that releases the GIL (NumPy `matmul`, Numba `nogil=True`, Cython, etc.). At that point we typically wouldn’t need threads either, because the GIL-free C kernel already saturates one core.

4. Where does Python multithreading alone beat sequential? I/O-bound and “GIL-released-by-the-library” workloads:

- **Network I/O fan-out.** HTTP requests with `requests`, database queries, file reads — the GIL is released during the kernel `recv/read`, so k threads can wait in parallel.
- **Disk I/O.** Reading many files from a fast SSD benefits because the kernel I/O is GIL-free.
- **Native C/Fortran kernels.** NumPy linear algebra (BLAS), `numpy.matmul`, `scipy.linalg.solve`, `numba.jit(nogil=True)`, image-processing libraries (Pillow, OpenCV) — all release the GIL inside the heavy work.
- **Compression / cryptography.** `hashlib`, `zlib`, OpenSSL bindings release the GIL.

For pure-Python CPU-bound code such as the TVB inner loop in `tvb_par.py`, only multiprocessing (separate interpreters, separate GILs) gives real parallelism — which is exactly why this lab walks us through that path.

LLM Usage Disclosure

I used Anthropic's Claude (Claude Code) for this assignment to:

- Help design the benchmark harness (`bench.py`, `run_all.sh`) and the chunk-size sweep methodology.
- Cross-check the GIL discussion against the official CPython documentation and `concurrent.futures` reference.
- Polish the prose of this report.

All measurements were collected by me on the assigned AWS instance, and all submitted code (`L2_Q6.py`, `L2_Q7.py`, `L2_Q8.py`) was written, validated, and inspected by me before submission.


```
bin/py-spy record -d 25 -o ../figures/pyspy_flame.svg --format flamegraph -- python3 bench.py par tvb192 15 128 --rep
```



Figure 2: py-spy top on tvb_par.py (TVB192, optimal chunk). The dominant frames are inside multiprocessing.connection and the binary serialiser, not the TVB kernels.